

Heterogeneous Architecture for Asynchronous Seamless Image Stitching

Hyerin Cho
VisionexT
South Korea

hyerin.cho@hanwha.com

Jin-Woo Kim
VisionexT
South Korea

jinwoo.kim@hanwha.com

Jinhong Park*
VisionexT
South Korea

jhp.park@hanwha.com

Jeong-Ho Woo
VisionexT
South Korea

jeongho.woo@hanwha.com

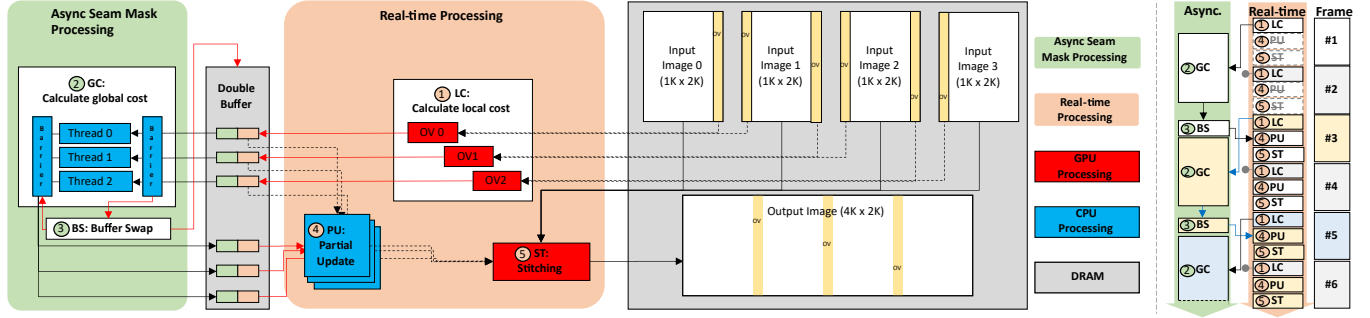


Figure 1: Overview of the Heterogeneous Architecture for Asynchronous Seamless Image Stitching

ACM Reference Format:

Hyerin Cho, Jin-Woo Kim, Jinhong Park, and Jeong-Ho Woo. 2024. Heterogeneous Architecture for Asynchronous Seamless Image Stitching. In *SIGGRAPH Asia 2024 Posters (SA Posters '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3681756.3697975>

1 Introduction

Image stitching, a technique for generating high-resolution images by combining multiple low-resolution input images, reveals the problem of unintended seams occurring caused by variations in image brightness or geometric misalignment. Existing methods primarily address this issue through two approaches: transition smoothing[Brown and Lowe 2007; Uyttendaele et al. 2001] and optimal seam finding[Chon et al. 2010; Efros and Freeman 2001; Kerschner 2001; Kwatra et al. 2003]. Transition smoothing aims to minimize stitching artifacts by reducing color and brightness differences in overlapping regions. However, this method can introduce undesirable artifacts such as ghosting or blurring. In contrast, optimal seam finding seeks to identify the seam with the least perceptible difference between two images, which is robust to ghosting and blurring issues.

To efficiently accelerate the computationally intensive optimal seam finding process in resource-constrained embedded SoC environments, we propose a heterogeneous CPU-GPU system. Real-time image stitching is accelerated on the GPU, while the analysis of

seam masks with minimal variations between frames, is accelerated on the CPU using an asynchronous approach. To mitigate quality degradation caused by outdated seam masks in the asynchronous approach, a partial update method was implemented. Experimental results show that applying the asynchronous approach resulted in a 113% increase in Frames Per Second (FPS) compared to the synchronous approach, while the seam cost, which is inversely proportional to quality, increased by 93%.

2 Our Approach

In the proposed architecture, the GPU performs image stitching and the CPU performs seam analysis. While GPUs are suited for image processing, making image stitching efficient, CPU-based seam analysis is computationally expensive. Consequently, a synchronous approach that performs seam analysis on every frame degrades GPU utilization. To improve this, as shown in Figure 1, we applied asynchronous method and double buffering, where the GPU continuously performs stitching based on the previous seam mask even while the CPU is analyzing the current seam. To improve the quality degradation caused by delayed seam mask, we applied a real-time partial update method.

2.1 Asynchronous Seam Mask Processing

A dynamic programming-based optimal seam finding algorithm [Efros and Freeman 2001] was used in this work to generate seam masks efficiently on CPUs with limited computational resources. The local costs required for seam mask generation are calculated by the GPU through accelerated gradient computations between adjacent pixels in the images. Gradient-based costs are robust to noise and are used to maintain consistent visual quality.

Subsequently, as shown in Figure 1, the CPU receives local costs for three overlapping regions (OV) from the GPU, and to enable simultaneous processing, multithreading with SIMD acceleration is

*Corresponding author

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1138-1/24/12

<https://doi.org/10.1145/3681756.3697975>

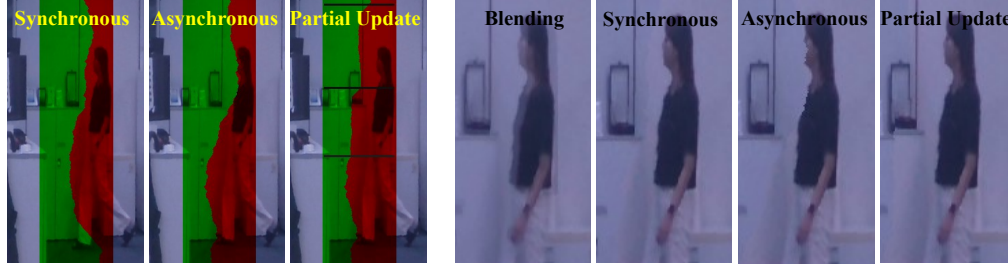


Figure 2: Qualitative evaluation results (left: seamline, right: original)

applied to compute the cumulative global cost. The optimal seam is then determined by backtracking from the minimum value.

2.2 Real-time Image Stitching

Image stitching, including lens distortion correction (LDC) and geometric projection, is accelerated in real-time on the GPU. In this work, to accelerate LDC and geometric projection for 8K resolution images with a grid size of 30×16 , an LUT size of 256×256 was employed, resulting in a 106.39% performance improvement. Furthermore, to reduce shader complexity caused by branching, separate shader programs were generated for overlap and non-overlap regions, leading to a 67% performance enhancement.

2.3 Partial Update

Seam masks generated asynchronously on the CPU can become outdated during real-time processing, potentially degrading the quality of seamless processing. To address this issue in real-time environments, a partial update method is applied only to the regions with a high probability of quality degradation based on the cost of each block within the seam mask. To predict the likelihood of quality degradation, the seam mask is divided into blocks, and the cost of each block is calculated based on the path of the outdated seam mask and the latest local cost. Only the worst case block that can be processed within a real-time threshold is updated. A penalty term was added to the cost function to maintain continuity between the partially updated and unchanged seam paths, addressing the seam path discontinuity problem.

3 Results and Conclusions

A real-time video environment was established by utilizing four consecutive 2K resolution images to generate a 4K resolution output video consisting of 3500 frames. Experiments were conducted on a system equipped with four 1.8GHz Cortex-A55 CPUs, a 1GHz ARM Mali-G610 MP4 GPU, and 4GB RAM, with the GPU limited to 1 core at 600MHz. To assess the performance benefits of the proposed asynchronous structure, we compared it to a synchronous structure and additionally evaluated the effectiveness of partial updates in mitigating quality degradation. FPS was used as the performance evaluation metric, while the total cost of the seam path used for stitching was utilized as the quality evaluation metric.

Table 1 represents a comparative analysis of the performance and quality of each proposed method. The asynchronous approach achieved a 113% improvement in FPS compared to the synchronous approach, but with a 93% increase in cost. Additionally, performance

Table 1: Quantitative experimental results

Method	Sync	Async	Async + Partial Update					
No. updates	N/A	0	1	2	3	4	5	6
FPS	40.1	85.6	71.5	63.1	54.7	50.3	45.9	42.2
[%]	100	213	178	157	137	126	114	105
Cost[%]	52	100	87	83	81	79	78	77
Delay[frames]	0	2	2	2	2	2	2	2

experiments on partial updates were conducted considering the trade-off between performance and quality by varying the number of update regions. As the number of updated regions increased from 1 to 6, FPS decreased from 85.6 to 42.2, and the cost decreased from 100% to 77%. A comparative analysis at 60 FPS revealed that performing partial updates twice resulted in a 57% increase in FPS relative to the synchronous approach and a 17% decrease in cost compared to the asynchronous approach. In Figure 2, synchronous approach and partial update performing twice show the improved image quality compared to the blending and asynchronous approach.

We propose a seamless image stitching method on a heterogeneous CPU-GPU system that achieves real-time 60fps at 4K resolution without ghosting in embedded environments. Experiments demonstrate improved performance with our asynchronous approach and enhanced image quality through partial updates. The proposed partial update offers a flexible trade-off between quality and performance, suitable for various applications including surveillance, remote sensing, and virtual reality.

References

- Matthew Brown and David G Lowe. 2007. Automatic panoramic image stitching using invariant features. *International journal of computer vision* 74 (2007), 59–73.
- Jaechoon Chon, Hyongsuk Kim, and Chun-Shin Lin. 2010. Seam-line determination for image mosaicking: A technique minimizing the maximum local mismatch and the global cost. *ISPRS Journal of Photogrammetry and Remote Sensing* 65, 1 (2010), 86–92.
- Alexei A. Efros and William T. Freeman. 2001. Image quilting for texture synthesis and transfer. In *Proceedings of the 28th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH '01)*. Association for Computing Machinery, New York, NY, USA, 341–346. <https://doi.org/10.1145/383259.383296>
- Martin Kerschner. 2001. Seamline detection in colour orthoimage mosaicking by use of twin snakes. *ISPRS journal of photogrammetry and remote sensing* 56, 1 (2001), 53–64.
- Vivek Kwatra, Arno Schödl, Irfan Essa, Greg Turk, and Aaron Bobick. 2003. Graphcut textures: Image and video synthesis using graph cuts. *Acm transactions on graphics (tog)* 22, 3 (2003), 277–286.
- Matthew Uyttendaele, Ashley Eden, and Richard Skeliski. 2001. Eliminating ghosting and exposure artifacts in image mosaics. In *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition. CVPR 2001*, Vol. 2. IEEE, II–II.